

Audio Song Classification from Humming and Whistling

GitHub Repository:

<https://github.com/oscar5198/audio-song-classification>

Portfolio Technical Report

Reproducible machine learning study using participant-independent evaluation

Selected model	Validation-selected Random Forest
Dataset	MLEnd Humming and Whistling II 800-file sample
Evaluation protocol	Participant-independent train/validation/test split
Repository outputs	Saved model, metrics, classification report, confusion matrix

Table of Contents

Table of Contents	2
Executive Summary.....	3
Introduction	3
Problem Statement.....	4
Project Contributions	4
Dataset	4
Methodology.....	5
Audio Preprocessing	5
Feature Extraction.....	6
Participant-Independent Splitting.....	6
Model Selection Rationale	6
Experimental Pipeline	7
Repository Architecture	8
Experimental Results	8
Error Analysis	10
Discussion.....	11
Strengths	11
Limitations	11
Practical Implications	12
Reproducibility	12
Lessons Learned	12
Future Work.....	12
Conclusion.....	12
References	13

Executive Summary

This portfolio report presents a reproducible machine learning study for recognizing songs from short humming and whistling recordings. The motivation is practical as well as methodological: query-by-humming systems are useful when a listener remembers a melody but not a title, yet human-produced melody fragments are noisy, inconsistent, and highly participant-dependent. The problem is framed as eight-class audio classification, where each input recording must be assigned to one of eight song labels.

The study uses the MLEnd Humming and Whistling II 800-file sample and emphasizes evaluation design. Audio files are normalized, resampled, and converted into fixed 10-second analysis windows. The final feature representation combines compact statistics from MFCCs, MFCC deltas, log-mel summaries, chroma features, dominant-frequency estimates, and spectral-shape descriptors. Candidate approaches include Logistic Regression, Random Forest, SVM, and CNN baselines, but model selection is validation-led under a participant-independent split.

The selected model is a Random Forest trained on compact combined features. It achieved validation accuracy 0.3583 and validation macro F1 0.3537; on the held-out participant-independent test split it achieved accuracy 0.2750 and macro F1 0.2629. The central finding is that honest participant-independent evaluation produces lower but more credible scores than leakage-prone evaluation. The project contribution is therefore not only a final classifier, but a reproducible repository that demonstrates corrected methodology, transparent error analysis, and a defensible end-to-end ML workflow.

Portfolio positioning: The repository is best understood as a rigorous small-data audio ML study: it values reproducibility, leakage control, and transparent reporting over inflated benchmark numbers.

Introduction

Humming and whistling song recognition is a compact version of a larger music information retrieval problem: identifying a musical work from an imperfect human memory trace. Unlike studio recordings, participant-generated clips can vary in key, tempo, rhythm, pitch accuracy, breath noise, microphone quality, recording duration, and expressive style. Two people may produce recognizably different versions of the same melody, while two different songs may share similar contour fragments in a short excerpt.

This makes the task scientifically interesting. A classifier must learn song-relevant structure without overfitting to the participant, device, or recording context. In small audio datasets, this distinction matters because random file-level splitting can accidentally place the same participant in training and test data. That leakage can make the task appear easier than it is, especially when vocal timbre, microphone response, or personal phrasing becomes a proxy for the label.

For this reason, reproducible evaluation is treated as a first-class part of the project. The repository records metadata, split assignments, model artifacts, metrics, classification reports, and figures so that the final result can be inspected and rerun. The report presents the final repository as a professional machine learning study rather than a one-off experiment.

Problem Statement

Given a short humming or whistling recording, the model must predict the corresponding song label from eight classes: Feeling, Friend, Happy, Married, Necessities, NewYork, RememberMe, and TryEverything. The evaluation objective is not only high accuracy, but credible generalization to unseen participants. The model is therefore trained, validated, and tested using participant-independent splits.

Project Contributions

Table 1. Summary of project contributions.

Contribution	What it adds
Reproducible machine learning pipeline	The project is organized as scripts for metadata construction, subject-aware splitting, feature extraction, training, and artifact generation.
Participant-independent evaluation	The split prevents the same participant from appearing in multiple partitions, reducing leakage from voice, device, and recording habits.
Comparison of multiple ML approaches	The repository compares classical linear, tree-based, kernel, and convolutional approaches while keeping the final selection validation-led.
Methodology correction	The final repository separates corrected evaluation from archived exploratory work and explains why participant leakage changes interpretation.
Transparent evaluation and error analysis	Metrics, per-class reports, confusion matrix outputs, and qualitative error explanations are saved and reported directly.

Dataset

The study uses the local MLEnd Humming and Whistling II 800-file sample. Filenames encode the participant ID, recording type, take number, and song label, allowing metadata to be generated deterministically. The older overlapping 400-file sample is not used for corrected evaluation.

Table 2. Dataset summary.

Dataset property	Value
Recordings	800

Dataset property	Value
Unique participants	187
Song classes	8
Recording types	400 humming, 400 whistling
Mean duration	21.47 seconds
Duration range	10.62 to 39.09 seconds

Table 3. Class distribution by participant-independent split.

Song label	Train	Validation	Test
Feeling	67	16	17
Friend	71	14	15
Happy	67	17	16
Married	72	14	14
Necessities	68	17	15
NewYork	70	15	15
RememberMe	72	14	14
TryEverything	73	13	14

Methodology

Audio Preprocessing

Each WAV file is loaded as mono floating-point audio. Stereo inputs are averaged across channels, integer amplitudes are scaled into floating-point range, and each signal is peak-normalized to reduce level differences between recordings. The pipeline then resamples audio to the project target sample rate and applies a fixed-duration analysis window.

For the final model, recordings are padded or truncated to 10 seconds. This choice standardizes feature dimensions while retaining enough signal for compact melodic and spectral summaries. It also avoids letting original recording duration become an uncontrolled feature.

Feature Extraction

The final representation is a combined handcrafted feature vector. MFCC and MFCC-delta statistics summarize timbral and short-term spectral envelope information; log-mel summaries capture coarse time-frequency energy patterns; chroma features approximate pitch-class distribution; dominant-frequency features provide a simple contour proxy; and spectral centroid, bandwidth, and rolloff summarize brightness and spectral spread.

These features are compact rather than sequence-preserving. That makes them appropriate for small-data classical machine learning, but it also explains a key limitation: they cannot fully represent detailed melodic ordering or long-range temporal structure.

Participant-Independent Splitting

Table 4. Participant-independent split summary.

Split	Recordings	Unique participants
Train	560	129
Validation	120	29
Test	120	29

The split is grouped by participant ID, so all recordings from a participant appear in only one partition. This design tests whether the learned mapping transfers to new people rather than memorizing participant-specific recording signatures.

Model Selection Rationale

Table 5. Model rationale.

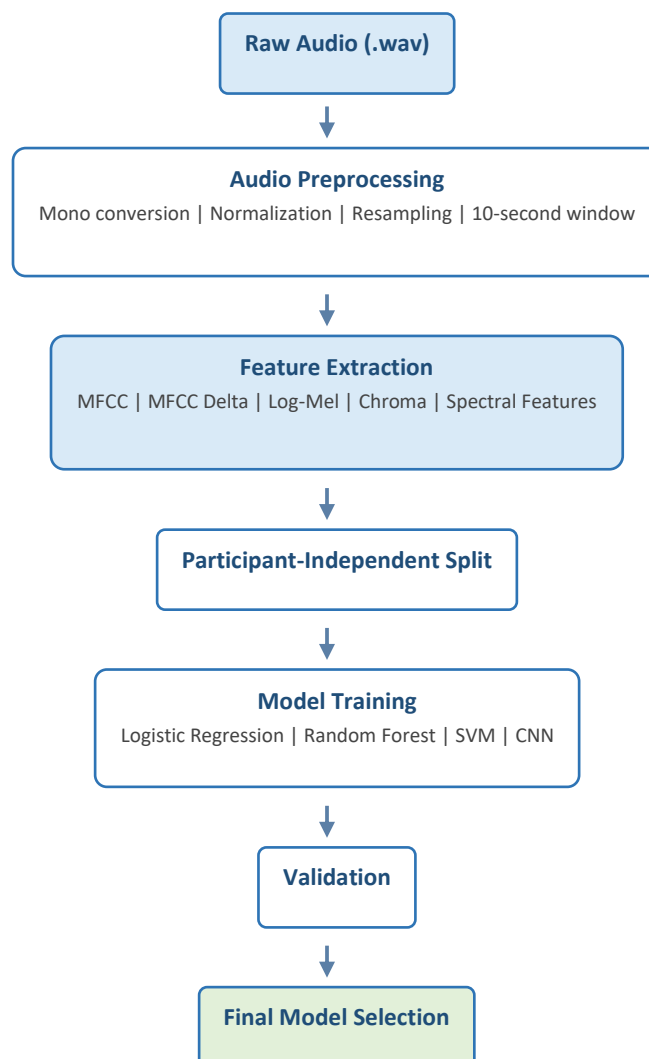
Model	Reason for inclusion
Logistic Regression	A linear baseline for evaluating whether compact or flattened features are linearly separable.
Random Forest	A nonlinear ensemble suited to mixed handcrafted features, small datasets, and irregular feature interactions [1].
SVM	A strong classical kernel baseline for compact feature spaces and nonlinear boundaries.

Model	Reason for inclusion
CNN	A time-frequency baseline for log-mel spectrogram inputs, included to test whether learned local filters improve performance.

The Random Forest became the final model because it was selected by validation performance after controlled diagnostics and then evaluated once on the held-out test split. Although another baseline produced a slightly higher observed test score, choosing it after seeing test results would make the final decision test-led. The validation-selected Random Forest is therefore the more defensible model for a reproducible portfolio study.

Experimental Pipeline

The final repository exposes the experiment as a repeatable pipeline rather than a manual notebook workflow. Each stage writes artifacts that can be inspected independently, which makes failures easier to diagnose and results easier to reproduce.



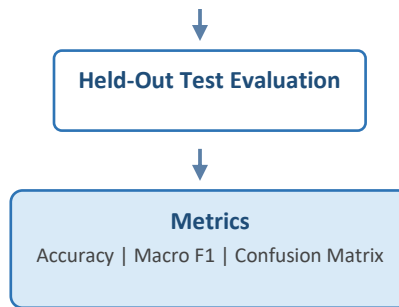


Figure 1. Editable experimental pipeline diagram for the final repository workflow.

Reproduction path: Run ``src/build_metadata.py``, ``src/create_splits.py``, and ``src/train_final_model.py`` from the repository root to regenerate metadata, split assignments, final metrics, model artifact, and the confusion matrix.

Repository Architecture

The repository separates raw data references, source code, saved results, model artifacts, figures, and archived exploratory work. This layout keeps the final portfolio pipeline visible while preserving earlier experiments for context.

Repository area	Purpose and key artifacts
data/	Raw audio location and dataset README MLEndHWII_sample_800/ kept local, not committed
src/	build_metadata.py create_splits.py features.py train_final_model.py
results/	metadata.csv splits.csv final_model_comparison.csv final_random_forest_metrics.csv classification_reports/final_random_forest_test.txt
models/ and figures/	final_random_forest.joblib confusion_matrix_final_random_forest.png
root files	README.md REPRODUCIBILITY.md requirements.txt

Figure 2. Repository architecture showing the final reproducible project layout.

Experimental Results

Table 6 reports the final comparison view from the repository. The selected Random Forest is highlighted because it was the validation-led final model. The table includes test scores for transparency, but test performance is not used retroactively to choose the final model.

Table 6. Final comparison table for evaluated models. The selected model is shaded.

Model	Features	Val. acc.	Test acc.	Macro F1	Role
Original Logistic Regression	Flattened MFCCs with original-style train augmentation	0.325	0.2833	0.2878	Comparator
Original Random Forest	Flattened MFCCs with original-style train augmentation	0.2833	0.2583	0.2557	Comparator
Original CNN	Log-mel spectrograms with original CNN architecture	0.2833	0.2083	0.144	Comparator
Selected Random Forest	Combined compact summaries: MFCC/delta/log-mel/chroma/pitch/spectral	0.3583	0.275	0.2629	Selected
SVM	Combined compact summaries: MFCC/delta/log-mel/chroma/pitch/spectral	0.3583	0.2333	0.2337	Comparator

Table 7. Final Random Forest validation and held-out test metrics.

Split	Accuracy	Macro precision	Macro recall	Macro F1	Weighted F1
Validation	0.3583	0.3618	0.3546	0.3537	0.3600
Test	0.2750	0.2581	0.2797	0.2629	0.2594

Validation was used for model selection because the test set should represent a final, unbiased estimate of generalization after the modelling decision has already been made. This discipline is especially important in small datasets, where a single test split can produce noisy model rankings.

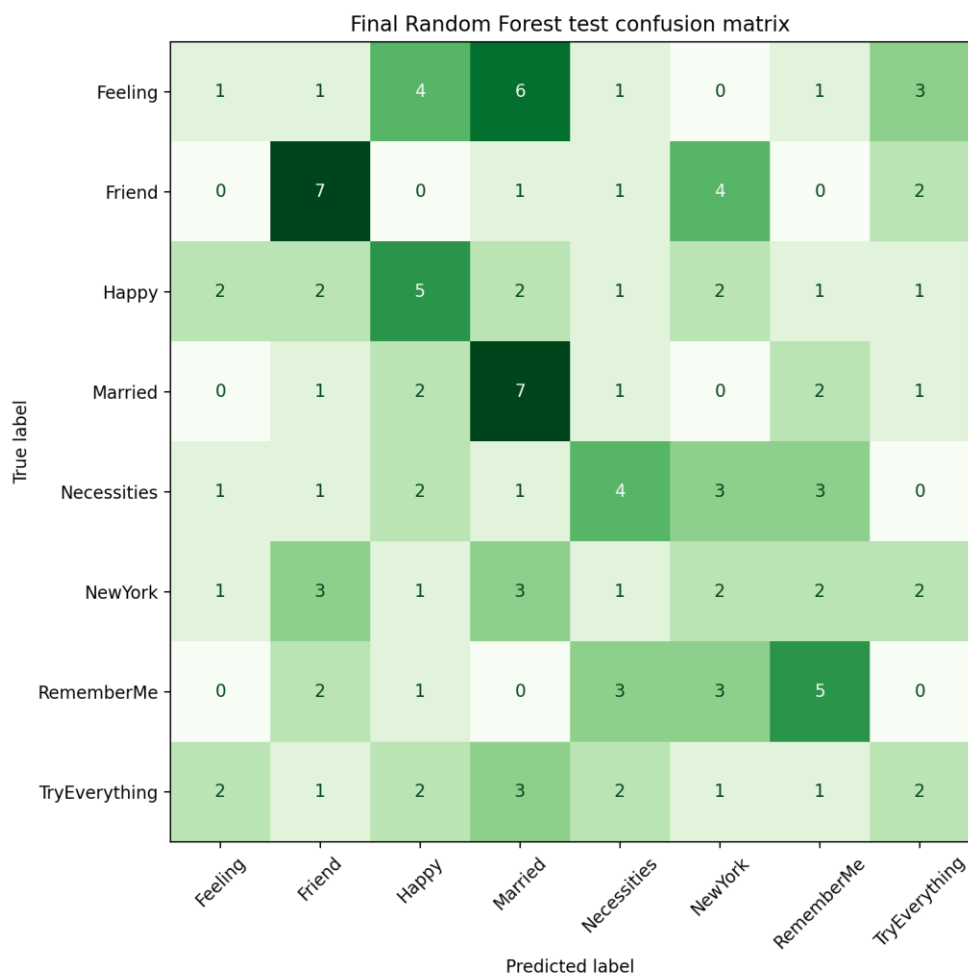


Figure 3. Confusion matrix for the selected Random Forest on the participant-independent test split.

Error Analysis

Table 8. Per-class test classification report for the selected Random Forest.

Class	Precision	Recall	F1	Support
Feeling	0.14	0.06	0.08	17
Friend	0.39	0.47	0.42	15
Happy	0.29	0.31	0.30	16
Married	0.30	0.50	0.38	14
Necessities	0.29	0.27	0.28	15

Class	Precision	Recall	F1	Support
NewYork	0.13	0.13	0.13	15
RememberMe	0.33	0.36	0.34	14
TryEverything	0.18	0.14	0.16	14

For the final Random Forest, the strongest classes by F1 are Friend, Married, and RememberMe. The weakest classes are Feeling, NewYork, and TryEverything. The easiest and hardest songs are not perfectly stable across model families, which suggests that different feature representations capture different parts of the humming and whistling signal.

Common confusions are musically plausible: short clips can share contour fragments, rhythm patterns, pitch ranges, or broad spectral shape even when the underlying songs differ. Because the final compact features summarize statistics over time, they can blur the ordering information that a listener would use to distinguish similar melodic phrases.

The CNN struggled because the corrected training set contains only 560 recordings, and participant variability is high. A convolutional model can exploit local time-frequency structure, but without enough diverse training data it may underfit, overfit, or learn recording-specific cues that do not transfer to unseen participants.

The Random Forest performed best among the validation-led candidates because it can model nonlinear interactions across heterogeneous handcrafted features without requiring the data volume expected by deeper models. Its advantage is still modest, which reinforces the main lesson: honest evaluation produces lower but more trustworthy scores.

Discussion

Strengths

- The final workflow is reproducible, script-based, and organized around saved artifacts.
- Participant-independent evaluation directly addresses leakage risk and makes the reported scores more credible.
- Multiple model families are compared, including linear, ensemble, kernel, and convolutional approaches.
- Negative results are preserved and interpreted rather than hidden.

Limitations

- The dataset is small for eight-class audio classification, especially after subject-aware splitting.
- The final feature representation loses detailed melodic ordering because it aggregates statistics over time.

- Validation and test sets contain 120 recordings each, so class-level conclusions should be treated cautiously.
- The CNN experiments are useful diagnostics but not a complete deep-learning exploration.

Practical Implications

A production query-by-humming system would likely need larger and more varied data, melody-aware representations, calibration, and robust abstention behavior for ambiguous clips. However, this repository demonstrates the foundation required before scaling: leakage-aware splitting, reproducible artifacts, and transparent evaluation.

Reproducibility

The repository records every major stage of the pipeline: generated metadata, split assignments, metrics, reports, figures, and the saved final model. This makes the project suitable for external review because a reader can trace a reported result back to a script and artifact.

Lessons Learned

- Data splitting can dominate apparent performance in small audio datasets.
- Validation-led selection is essential when test scores are noisy.
- Classical ML remains valuable when data are limited and features are engineered carefully.
- Transparent limitations make a portfolio project stronger, not weaker.

Future Work

- Add subject-aware cross-validation for more stable model estimates.
- Use a pinned audio stack with a dedicated library such as librosa.
- Explore pitch contours, interval sequences, dynamic time warping, and sequence models that preserve melodic order.
- Add confidence calibration and abstention for ambiguous recordings.
- Revisit neural models with stronger regularization, more data, transfer learning, or validation-constrained architecture search.

Conclusion

This project built an end-to-end, reproducible audio classification pipeline for recognizing songs from humming and whistling recordings. It converts local WAV files into metadata, participant-independent splits, compact audio features, trained models, saved metrics, a classification report, and a final confusion matrix.

The main technical lesson is that evaluation protocol matters as much as model choice. Under participant-independent testing, the task is substantially harder than leakage-prone evaluation suggests. The selected Random Forest is valuable because it is chosen through a defensible validation process, works with compact handcrafted features, and provides a transparent final baseline for future research.

The repository is valuable because it demonstrates research judgment: correcting methodology, preserving negative results, reporting limitations honestly, and leaving a reproducible foundation for future work. Strong next directions include subject-aware cross-validation, melody-preserving features, calibrated uncertainty, and larger-scale neural or sequence-based models.

References

- [1] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- [2] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357-366, 1980.
- [3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [4] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261-272, 2020.
- [5] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273-297, 1995.
- [6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436-444, 2015.
- [7] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [8] MLEnd Humming and Whistling II dataset documentation and local 800-file sample used in this repository.